DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

Índice general

Resumen I

1. Introducción general 1
1.1. Contexto del trabajo 1
1.2. Estado del arte 2
1.3. Motivación 3
1.4. Objetivos y alcance 3

2. Introducción específica 7
2.1. Técnicas de procesamiento de datos 7
2.1.1. Detección y tratamiento de outliers 7
2.1.2. Escalado de variables 8
2.1.3. Balanceo de dataset 9
2.1.4. Selección de features 9
2.2. Modelos de inteligencia artificial 9
2.3. Optimización de hiperparámetros 10
2.4. Herramientas - Tracking server 10
2.5. Herramientas para automatización 10
2.6. Desarrollo de API y herramientas para su consumo 11
2.7. Uso de contenedores 11

Bibliografía 13

v

Índice general

Resumen I

1. Introducción general 1
1.1. Contexto del trabajo 1
1.2. Estado del arte 2
1.3. Motivación 3
1.4. Objetivos y alcance 3

2. Introducción específica 7
2.1. Técnicas de procesamiento de datos 7
2.1.1. Detección y tratamiento de outliers 7
2.1.2. Escalado de variables 8
2.1.3. Balanceo de dataset 9
2.1.4. Selección de features 9
2.2. Modelos de inteligencia artificial 9
2.3. Optimización de hiperparámetros 10
2.4. Herramientas - Tracking server 10
2.5. Herramientas para automatización 10
2.6. Desarrollo de API y herramientas para su consumo 11
2.7. Uso de contenedores 11

3. Diseño e implementación 13
3.1. Flujo de trabajo 13
3.1.1. Etapas del flujo de trabajo 13
3.1.2. Implementación del flujo de trabajo 14
3.2. Arquitectura del sistema 14
3.2.1. Arquitectura del ambiente local 14
3.2.2. Arquitectura del ambiente dockerizado 15
3.3. Preprocesamiento de datos 16
3.3.1. División del dataset 17
3.3.2. Tratamiento de nulos y outliers 17
3.3.3. Escalado de datos 19
3.3.4. Balanceo del dataset 20
3.3.5. Selección de features 21
3.4. Optimización y entrenamiento de modelos 21
3.4.1. Pipelines y modelos 21
3.4.2. Cross validation e implementación durante optimización 22
3.4.3. Modelo Logistic Regression 22
3.4.4. Modelo SVM 22
3.4.5. Modelo XGBoost 22
3.5. Tracking server 22
3.6. Automatización mediante Airflow 22

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

3.7. Servicio web 22

Bibliografía 23

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

Índice de figuras

Índice de figuras

3.1. Flujo de trabajo utilizado. 13
3.2. Arquitectura del ambiente local. 15
3.3. Arquitectura del ambiente dockerizado. 16
3.4. Transformación de Yeo-Johnson sobre la columna Total Asset Turnover. 18
3.5. Imputación por mediana sobre la columna Cash Flow Per Share. . . 18
3.6. Standard scaler aplicado a Total expense/assets. 19
3.7. Standard scaler aplicado a Retained Earnings to Total Assets. 19
3.8. Desbalance de la columna Bankrupt? en el dataset de train. 20

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

Índice de tablas

2.1. Técnicas utilizadas 7

Índice de tablas

2.1. Técnicas utilizadas 7
3.1. Técnicas utilizadas 20

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

Capítulo 1

Introducción general

En este capítulo se comienza explicando al lector en qué consiste la quiebra de una empresa, su diferencia con la insolvencia, algunos de las posibles causas de la misma, sus repercusiones y por qué resulta de gran valor predecir este tipo de quiebras. Luego, se detalla el estado del arte en lo que respecta a la predicción de quiebra, remarcando las ventajas y desventajas de las soluciones actuales. Posteriormente, se explican las motivaciones detrás de este trabajo. Finalmente, se describen los objetivos y el alcance del trabajo.

1.1. Contexto del trabajo

La quiebra es un proceso legal y económico en el que entra una empresa cuando esta es incapaz de cumplir con sus obligaciones financieras hacia sus acreedores a medida que vencen, por lo que busca protección o resolución bajo procedimientos legales. Este proceso puede resultar en la reorganización, es decir, en la continuación de operaciones bajo supervisión, o en una liquidación, que consiste en la venta los activos de la empresa junto con su posterior cierre.

Es importante marcar la diferencia entre una empresa insolvente y una en quiebra. La insolvencia es una condición necesaria pero no suficiente para que una empresa entre en quiebra. Una empresa insolvente se caracteriza por tener más activos que pasivos, pero es incapaz de pagar sus deudas a medida que vencen. Pero, una potencial liquidación de estos activos permitiría generar ingresos para pagar a los acreedores. Es decir, la insolvencia hace referencia a una condición financiera, la cual pueder ser temporal e incluso reversible. Por su parte, la quiebra consiste en un proceso legal, al que se llega por la persistencia de la insolvencia. Una característica de las empresas en quiebra es que poseen un patrimonio neto negativo, es decir, poseen menos activos que pasivos, por lo cual sus acreedores no recibirían el pago de sus deudas, incluso si la empresa fuese liquidada [1].

Generalmente, la quiebra es el resultado de una acumulación de factores como problemas de flujo de caja, exceso de deuda, mala gestión operativa, saturación del mercado, cambios regulatorios o incluso crisis económicas.

Las repercusiones de la quiebra no solo se ven en el ámbito económico, sino que también en los ámbitos social y legal:

- Económicamente, la quiebra de una empresa podría perjudicar a algunos de sus acreedores, ya que no podrían recibir el pago de sus deudas. Esto a su vez podría generar un efecto dominó sobre estos, provocándoles su insolvencia y/o quiebra. Puede también reducir la productividad económica

Capítulo 1

Introducción general

En este capítulo se introduce el concepto de quiebra empresarial, su distinción con la insolvencia, sus principales causas y repercusiones, y la relevancia de anticiparla mediante modelos predictivos. A continuación, se presenta el estado del arte en predicción de quiebra, junto con las ventajas y limitaciones de las soluciones existentes. Por último, se expone las motivación, el objetivo y el alcance del presente trabajo.

1.1. Contexto del trabajo

La quiebra es un proceso legal y económico en el que entra una empresa cuando es incapaz de cumplir con las obligaciones financieras hacia sus acreedores a medida que vencen, por lo que busca protección o resolución bajo procedimientos legales. Este proceso puede resultar en la reorganización, es decir, en la continuación de operaciones bajo supervisión, o en una liquidación, que consiste en la venta los activos de la empresa junto con su posterior cierre.

Es importante marcar la diferencia entre una empresa insolvente y una en quiebra. La insolvencia es una condición necesaria pero no suficiente para que una empresa entre en quiebra. Una empresa insolvente se caracteriza por tener más activos que pasivos, pero es incapaz de pagar sus deudas a medida que vencen. Sin embargo, una potencial liquidación de estos activos permitiría generar ingresos para pagar a los acreedores. Es decir, la insolvencia hace referencia a una condición financiera que pueder ser temporal e incluso reversible. Por su parte, la quiebra consiste en un proceso legal, al que se llega por la persistencia de la insolvencia. Una característica de las empresas en quiebra es que poseen un patrimonio neto negativo, es decir, poseen menos activos que pasivos, por lo que sus acreedores no recibirían el pago de sus deudas, incluso si la empresa fuese liquidada [1].

Generalmente, la quiebra es el resultado de una acumulación de factores como problemas de flujo de caja, exceso de deuda, mala gestión operativa, saturación del mercado, cambios regulatorios o incluso crisis económicas.

Las repercusiones de la quiebra no solo se ven en el ámbito económico, sino que también en los ámbitos social y legal:

- Económicamente, la quiebra de una empresa podría perjudicar a algunos de sus acreedores, ya que no podrían recibir el pago de sus deudas. Esto a su vez podría generar un efecto dominó sobre estos, provocándoles su insolvencia y/o quiebra. Puede también reducir la productividad económica

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

de una región y el Producto Bruto Interno (PBI), dependiendo del tamaño e influencia de la empresa afectada.

- Socialmente, el desempleo es la consecuencia más directa de una quiebra. Dependiendo de la cantidad de gente afectada, se podría incluso hablar de una recesión a nivel local.
- Legalmente, los procedimientos pueden durar meses o años, lo cual consume recursos que se podrían haber utilizado para pagar deudas. También puede presentar ciertos conflictos de interés entre los acreedores y otros interesados, por ejemplo, al definir quien tiene prioridad sobre el cobro de sus obligaciones.

Es por esto que la predicción de quiebra de una empresa resulta fundamental para distintos interesados. Inversionistas y accionistas podrían decidir si comprar o vender acciones ante una posible pérdida de valor; entidades bancarias evaluarían la calidad crediticia de una empresa y tendrían mejor información para determinar si les otorgan un préstamo o no; algunos proveedores podrían ajustar condiciones de crédito y evitar impagos; los gobiernos podrían diseñar políticas de rescate y apoyo; e incluso la propia gerencia de una empresa podría utilizarla como un sistema de alerta temprana, a fin de implementar medidas de reestructuración o de reducción de costos a fin de evitar la quiebra.

1.2. Estado del arte

Existen diversos estudios, técnicas, mecanismos e incluso soluciones de software que permiten predecir la quiebra de empresas.

Hay estudios que buscan encontrar soluciones generales a esta problemática. Por ejemplo, la fórmula matemática *z-score* de Altman [2] utiliza cinco ratios financieros para obtener la probabilidad de que una empresa quiebre en los próximos dos años. Otras soluciones más modernas, como la propuesta en el paper *Bankruptcy Prediction Using Machine Learning and Data Preprocessing Techniques* [3], proponen el uso de modelos de *machine learning* y técnicas de preprocesamiento de datos para solucionar este problema.

Otros estudios se basan en el análisis de un dataset en particular. Por ejemplo, para el dataset de *Taiwanese Bankruptcy Prediction* [4] hay estudios como *Comprehensive Evaluation of Bankruptcy Prediction in Taiwanese Firms Using Multiple Machine Learning Models*, en donde se exploran distintos modelos de *machine learning*; mientras que en otros, como en *Undersampling bankruptcy prediction: Taiwan bankruptcy data* [5], se estudian los resultados de aplicar ciertas técnicas de preprocesamiento sobre el dataset, más precisamente, técnicas de *undersampling*.

En cuanto a soluciones de software, existen herramientas como *Gini Machine*[1], una plataforma de análisis predictivo basada en aprendizaje automático, y diseñada bajo el enfoque *no-code*.

Si bien el estado del arte para la predicción de quiebra de empresas presenta diferente soluciones, se observan ciertas desventajas en la mayoría de ellas.

[1] https://ginimachine.com/

de una región y el Producto Bruto Interno (PBI), dependiendo del tamaño e influencia de la empresa afectada.

- Socialmente, el desempleo es la consecuencia más directa de una quiebra. Dependiendo de la cantidad de gente afectada, se podría incluso hablar de una recesión a nivel local.
- Legalmente, los procedimientos pueden durar meses o años, y esto consume recursos que se podrían haber utilizado para pagar deudas. También puede presentar ciertos conflictos de interés entre los acreedores y otros interesados, por ejemplo, al definir quién tiene prioridad sobre el cobro de sus obligaciones.

Es por esto que la predicción de quiebra de una empresa resulta fundamental para distintos interesados. Inversionistas y accionistas podrían decidir si comprar o vender acciones ante una posible pérdida de valor; entidades bancarias evaluarían la calidad crediticia de una empresa y tendrían mejor información para determinar si les otorgan un préstamo o no; algunos proveedores podrían ajustar condiciones de crédito y evitar impagos; los gobiernos podrían diseñar políticas de rescate y apoyo; e incluso la propia gerencia de una empresa podría utilizarla como un sistema de alerta temprana, a fin de implementar medidas de reestructuración o de reducción de costos a fin de evitar la quiebra.

1.2. Estado del arte

Existen diversos estudios, técnicas, mecanismos e incluso soluciones de software que permiten predecir la quiebra de empresas.

Hay estudios que buscan encontrar soluciones generales a esta problemática. Por ejemplo, la fórmula matemática *z-score* de Altman [2] utiliza cinco ratios financieros para obtener la probabilidad de que una empresa quiebre en los próximos dos años. Otras soluciones más modernas, como la propuesta en el paper *Bankruptcy Prediction Using Machine Learning and Data Preprocessing Techniques* [3], proponen el uso de modelos de *machine learning* y técnicas de preprocesamiento de datos para solucionar este problema.

Otros estudios se basan en el análisis de un dataset en particular. Por ejemplo, para el dataset de *Taiwanese Bankruptcy Prediction* [4] hay estudios como *Comprehensive Evaluation of Bankruptcy Prediction in Taiwanese Firms Using Multiple Machine Learning Models*, en donde se exploran distintos modelos de *machine learning*; mientras que en otros, como en *Undersampling bankruptcy prediction: Taiwan bankruptcy data* [5], se estudian los resultados de aplicar ciertas técnicas de preprocesamiento sobre el dataset, más precisamente, técnicas de *undersampling*.

En cuanto a soluciones de software, existen herramientas como Gini Machine [6], una plataforma de análisis predictivo basada en aprendizaje automático, y diseñada bajo el enfoque *no-code*.

Si bien el estado del arte para la predicción de quiebra de empresas presenta diferente soluciones, se observan ciertas desventajas en la mayoría de ellas.

Por ejemplo, en el caso de los estudios y papers, estos son útiles porque describen qué métodos, qué técnicas de procesamiento y qué modelos les permitieron

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

Por ejemplo, en el caso de los estudios y papers, estos son útiles porque describen qué métodos, qué técnicas de procesamiento y qué modelos les permitieron realizar predicciones y obtener buenos resultados. Sin embargo, ninguno presenta dichos modelos mediante una herramienta de acceso al público. Es decir que, si un interesado desea realizar predecir la quiebra de una empresa, este no solo deberá replicar los pasos explicados en el paper, sino que también deberá recolectar información necesaria para generar un set de datos que se adecue a sus necesidades. A su vez, esto imposibilita el acceso a usuarios no expertos en áreas técnicas, tales como matemática, ciencia de datos y/o *machine learning*.

Por su parte, *Gini Machine* resuelve algunas de las problemáticas mencionadas, ya que es una solución *no-code*, es decir, es de uso fácil para aquellos usuarios sin conocimientos programación, de estadística y de *machine learning*. Sin embargo, esta herramienta presenta ciertas desventajas, tales como un alto costo de uso, requerir que el usuario entienda qué datos son importantes para armar un buen dataset, e incluso requiere que la cantidad de datos presentados para armar un modelo sean como mínimo 1000 registros [2].

1.3. Motivación

Tal como se expresó en las secciones anteriores, la predicción de quiebra de una empresa representa información de gran valor para distintos actores del mercado financiero. Acceder a esta información en tiempo y forma les permitirá a bancos, compañías aseguradoras, fondos de inversión, consultoras especializadas en riesgo crediticio, acreedores e incluso a gerentes de empresas tomar mejores decisiones frente a una eventual quiebra de una empresa de su interés.

Si bien hoy existen diferentes soluciones a estas problemáticas, las cuales abarcan desde papers hasta soluciones de software, ninguna de estas ofrece una solución de uso fácil para cualquier tipo de usuario, especialmente para aquellos que no poseen conocimientos técnicos en áreas de matemática, estadística, *machine learning* o incluso programación.

Esto constituye la motivicación principal de este trabajo: la creación y la puesta en marcha de un sistema que permita predecir si una empresa entra en quiebra, priorizando que este sea de fácil acceso y uso para usuarios poco experimentados.

1.4. Objetivos y alcance

El objetivo de este trabajo consiste en satisfacer las necesidades de distintos actores del mercado financiero respecto a la predicción de quiebra de una empresa. Para ello, se diseña, se crea y se expone un sistema que permita realizar este tipo de predicciones. Este sistema se publica mediante un servicio web, a modo que sea de fácil acceso para los interesados. Respecto al diseño, el sistema requiere únicamente por parte del usuario el ingreso de los datos de una empresa de su interés. De esta forma se eliminan las desventajas presentes en soluciones actuales, tales como el ingreso de gran cantidad de datos para realizar predicciones, como así también tener conocimiento a nivel técnico de los datos presentados.

Para lograr este objetivo, se define el siguiente alcance:

[2] https://theresanaiforthat.com/ai/ginimachine/

realizar predicciones y obtener buenos resultados. Sin embargo, ninguno de estos trabajos expone sus modelos mediante una herramienta de acceso público. Reproducir los resultados exige no solo dominio técnico en áreas como ciencia de datos o *machine learning*, sino también la recolección y preparación de datos propios. Esto resulta prohibitivo para todos aquellos usuarios no especializados que podrían beneficiarse de estas herramientas.

Por su parte, Gini Machine resuelve algunas de las problemáticas mencionadas, ya que es una solución *no-code*, es decir, es de uso fácil para aquellos usuarios sin conocimientos programación, de estadística y de *machine learning*. Sin embargo, esta herramienta presenta ciertas desventajas, tales como un alto costo de uso, requerir que el usuario entienda qué datos son importantes para armar un buen dataset, e incluso requiere que la cantidad de datos presentados para armar un modelo sean como mínimo 1000 registros [7].

1.3. Motivación

Tal como se expresó en las secciones anteriores, la predicción de quiebra de una empresa representa información de gran valor para distintos actores del mercado financiero. Contar con esta información de manera oportuna les permitirá a bancos, aseguradoras, fondos de inversión y otros actores del sistema financiero tomar decisiones informadas frente a la inminente quiebra de una compañía bajo análisis.

Si bien hoy existen diferentes soluciones a estas problemáticas, que abarcan desde papers hasta soluciones de software, ninguna de estas ofrece una solución de uso fácil para cualquier tipo de usuario, especialmente para aquellos que no poseen conocimientos técnicos en áreas de matemática, estadística, *machine learning* o incluso programación.

Esto constituye la motivicación principal de este trabajo: la creación y la puesta en marcha de un sistema que permita predecir si una empresa entra en quiebra, priorizando que este sea de fácil acceso y uso para usuarios poco experimentados.

1.4. Objetivos y alcance

El objetivo de este trabajo consiste en satisfacer las necesidades de distintos actores del mercado financiero respecto a la predicción de quiebra de una empresa. Para ello, se diseña, se crea y se expone un sistema que permita realizar este tipo de predicciones. Este sistema se publica mediante un servicio web, de modo que sea de fácil acceso para los interesados. Respecto al diseño, el sistema requiere únicamente el ingreso de los datos de una empresa de su interés. De esta forma se eliminan las desventajas presentes en soluciones actuales, tales como el ingreso de gran cantidad de datos para realizar predicciones, como así también tener conocimiento a nivel técnico de los datos presentados.

Para lograr este objetivo, se definió el siguiente alcance:

- Identificar el set de datos para el problema: en este caso, se utiliza el dataset de origen público *Taiwanese Bankruptcy Prediction* [4].

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

- Definir el set de datos para el problema: en este caso, se utiliza el dataset de origen público *Taiwanese Bankruptcy Prediction* [4].
- Conducir un análisis exploratorio de datos: estudiar el dataset en cuestión es de vital importancia para conocer sus características, distribuciones, y potenciales errores a corregir.
- Realizar un preprocesamiento de datos: aplicar distintas técnicas de eliminación de duplicados, corrección de datos faltantes, corrección de *outliers*, balanceo de datos, selección de *features* más importantes, entre otras, es de suma importancia para mejorar la calidad del dataset.
- Implementar modelos de *machine learning*: es necesario implementar técnicas robustas basadas en *machine learning*, tales como SVM [6], logistic regression [7] y XGBoost [8], que permitan realizar tareas de clasificación. De esta manera, se presentan resultados de calidad al usuario.
- Optimizar hiperparámetros de modelos de *machine learning*: es fundamental refinar detalles relacionados a la configuración de los modelos empleados, a fin que la calidad de los resultados sean superiores.
- Obtener métricas y gráficos de cada modelo: estos les otorgan información extra a los usuarios respecto al contexto de sus predicciones.
- Comparar modelos y seleccionar al más optimo: es fundamental que el sistema le permita al usuario final realizar predicciones mediante el mejor modelo posible. Por ello, se comparan los distintos modelos entrenados y optimizados, a fin de encontrar al mejor de ellos.
- Seguimiento de resultados: al entrenar y comparar distintos modelos, es necesario hacer un seguimiento y una persistencia de ellos, como así también de sus métricas y gráficos. Esto facilita la consulta de esta información en distintas etapas, como así también su disposición para las consultas de los usuarios.
- Automatizar etapas: es de suma importancia que las etapas de análisis, optimización, entrenamiento y seguimiento de modelos sean automatizadas. Esto agiliza el ciclo de vida de la aplicación, permitiendo que las mejoras aplicadas al sistema sean publicadas de forma rápida y sencilla para el consumo de los usuarios.
- Crear un servicio web: mediante una API web, un usuario puede realizar predicciones de quiebra al proveer los datos de una empresa en particular. También puede consultar métricas y gráficas de los modelos involucrados, incluyendo al mejor de ellos.

Por otra parte, dentro de las actividades que quedaron excluidas del alcance, se pueden mencionar:

- Despliegue online: el trabajo realizado se expone como un servicio web, el cual se puede ejecutar de manera local. Es decir, no será desplegado en plataformas de *cloud computing*, tales como Azure [9], AWS [10], Google Cloud [11], entre otras.
- Creación de interfaz web: el trabajo se limita desde la definición de un dataset, hasta la publicación de un servicio web que permite realizar predicciones de quiebra de una empresa. No se desarrollará un sitio web para

4 Capítulo 1. Introducción general

- Conducir un análisis exploratorio de datos: estudiar el dataset en cuestión es de vital importancia para conocer sus características, distribuciones, y potenciales errores a corregir.
- Realizar un preprocesamiento de datos: aplicar distintas técnicas de eliminación de duplicados, corrección de datos faltantes, corrección de *outliers*, balanceo de datos, selección de *features* más importantes, entre otras, que son de suma importancia para mejorar la calidad del dataset.
- Implementar modelos de *machine learning*: es necesario implementar técnicas robustas, tales como SVM [8], logistic regression [9] y XGBoost [10], que permitan realizar tareas de clasificación. De esta manera, se presentan resultados de calidad al usuario.
- Optimizar hiperparámetros de modelos de *machine learning*: es fundamental refinar detalles relacionados a la configuración de los modelos empleados, a fin que la calidad de los resultados sean superiores.
- Obtener métricas y gráficos de cada modelo: estos les otorgan información extra a los usuarios respecto al contexto de sus predicciones.
- Comparar modelos y seleccionar el más optimo: es fundamental que el sistema le permita al usuario final realizar predicciones mediante el mejor modelo posible. Por ello, se comparan los distintos modelos entrenados y optimizados, a fin de encontrar al mejor de ellos.
- Seguimiento de resultados: al entrenar y comparar distintos modelos, es necesario hacer un seguimiento y una persistencia de ellos, como así también de sus métricas y gráficos. Esto facilita la consulta de esta información en distintas etapas, como así también su disposición para las consultas de los usuarios.
- Automatizar etapas: es de suma importancia que las etapas de análisis, optimización, entrenamiento y seguimiento de modelos sean automatizadas. Esto agiliza el ciclo de vida de la aplicación, permitiendo que las mejoras aplicadas al sistema sean publicadas de forma rápida y sencilla para el consumo de los usuarios.
- Crear un servicio web: mediante una API web, un usuario puede realizar predicciones de quiebra al proveer los datos de una empresa en particular. También puede consultar métricas y gráficas de los modelos involucrados, incluyendo al mejor de ellos.

Por otra parte, dentro de las actividades que quedaron excluidas del alcance, se pueden mencionar:

- Despliegue online: el trabajo realizado se expone como un servicio web que se puede ejecutar de manera local. Es decir, no fue desplegado en plataformas de *cloud computing*, tales como Azure [11], AWS [12], Google Cloud [13], entre otras.
- Creación de interfaz web: el trabajo abarca desde la definición de un dataset, hasta la publicación de un servicio web que permite realizar predicciones de quiebra de una empresa. No se desarrolló un sitio web para consumir este servicio. No obstante, el servicio podrá ser utilizado mediante otras herramientas existentes, tales como cURL [14].

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

consumir este servicio. No obstante, el servicio podrá ser utilizado mediante otras herramientas existentes, tales como cURL [12].

- Comercialización y distribución: el trabajo se limita a la creación y despliegue local del sistema descrito, más no a su comercialización y/o distribución.

1.4. Objetivos y alcance 5

- Comercialización y distribución: el trabajo se limita a la creación y despliegue local del sistema descrito, más no a su comercialización y/o distribución.

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

Capítulo 2

Introducción específica

En este capítulo se comienza detallando las técnicas de procesamiento de datos utilizadas en el trabajo. Luego, se describen los modelos de *machine learning* entrenados. Posteriormente, se describen las distintas herramientas utilizadas para hacer seguimiento de modelos, para automatizar el procesamiento de datos y entrenamiento de modelos, como así también de aquellas herramientas empleadas para consumir y testear APIs. Finalmente, se concluye el capítulo detallando el por qué se decidió utilizar contenedores de docker.

2.1. Técnicas de procesamiento de datos

En este trabajo se emplearon distintas técnicas de procesamiento de datos, las cuales permitieron detectar y tratar *outliers*, escalar variables, balancear el dataset y seleccionar features. En la tabla 2.1 se presentan las técnicas utilizadas.

TABLA 2.1. Técnicas utilizadas en el procesamiento de datos.

Técnica	Utilizada para
IQR	Detección de outliers
Imputación por media	Tratamiento de outliers
Imputación por mediana	Tratamiento de outliers
Imputación por moda	Tratamiento de outliers
Imputación por raíz cuadrada	Tratamiento de outliers
Imputación de Yeo-Johnson [13]	Tratamiento de outliers
Standard scaler	Escalado de variables
SMOTE	Balanceo de dataset
SMOTE-ENN	Balanceo de dataset
ADASYN	Balanceo de dataset
Información mutua	Selección de features

2.1.1. Detección y tratamiento de outliers

En este tipo de trabajo, en donde se entrenan modelos de modelos de *machine learning*, suelen utilizarse datasets con una gran cantidad de variables (columnas) y datos (registros).

Las variables de estos datasets suelen presentar distintos tipos de distribuciones matemáticas, las cuales se analizan a fin de descubrir algunas de sus características. Entre estas se encuentran los valores atípicos u *outliers*, cuya presencia no solo afecta a la propia distribución, sino también al rendimiento de los modelos.

Capítulo 2

Introducción específica

En este capítulo se comienza detallando las técnicas de procesamiento de datos utilizadas en el trabajo. Luego, se describen los modelos de *machine learning* entrenados. Posteriormente, se describen las distintas herramientas utilizadas para hacer seguimiento de modelos, para automatizar el procesamiento de datos y entrenamiento de modelos, como así también de aquellas empleadas en el desarrollo de APIs y contenedores.

2.1. Técnicas de procesamiento de datos

En este trabajo se emplearon distintas técnicas de procesamiento de datos, que permitieron detectar y tratar *outliers*, escalar variables, balancear el dataset y seleccionar features. En la tabla 2.1 se presentan las técnicas utilizadas.

TABLA 2.1. Técnicas utilizadas en el procesamiento de datos.

Técnica	Utilizada para
IQR	Detección de *outliers*
Imputación por media	Tratamiento de *outliers*
Imputación por mediana	Tratamiento de *outliers*
Imputación por moda	Tratamiento de *outliers*
Imputación por raíz cuadrada	Tratamiento de *outliers*
Imputación de Yeo-Johnson [15]	Tratamiento de *outliers*
Standard scaler	Escalado de variables
SMOTE	Balanceo de dataset
SMOTE-ENN	Balanceo de dataset
ADASYN	Balanceo de dataset
Información mutua	Selección de features

2.1.1. Detección y tratamiento de outliers

En este tipo de trabajo, en el que se entrenan modelos de modelos de *machine learning*, suelen utilizarse datasets con una gran cantidad de variables (columnas) y datos (registros).

Las variables de estos datasets suelen presentar distintos tipos de distribuciones matemáticas, que deben ser analizadas a fin de descubrir sus características. Entre estas se encuentran los valores atípicos u *outliers*, cuya presencia no solo afecta a la propia distribución, sino también al rendimiento de los modelos.

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

Por este motivo, uno de los procesamientos que se implementan en este trabajo es el de detectar y tratar *outliers*.

Para detectarlos, se utiliza la técnica de IQR o rango intercuartil [14]. Esta mide la dispersión del 50 % central de los datos, es decir, aquellos que se encuentran entre el percentil 25 (cuartil 1 o Q1) y el percentil 75 (cuartil 3 o Q3), y se define mediante la ecuación 2.1.

$$IQR = Q3 - Q1 \tag{2.1}$$

Una vez definido el rango intercuartil, se consideran outliers a aquellos valores que se encuentran 1.5 veces el IQR debajo del cuartil 1, o 1.5 veces el IQR sobre el cuartil 3, tal como se define en la ecuación 2.2.

Una vez detectados los outliers, se imputan mediante distintas técnicas, entre las que se encuentran:

- Imputación por media: reemplaza los valores atípicos con el valor medio de la distribución.
- Imputación por mediana: reemplaza los valores atípicos con la mediana de la distribución.
- Imputación por moda: reemplaza los valores atípicos con la moda de la distribución.
- Transformación por raíz cuadrada: aplica la función de raíz cuadrada sobre la columna entera.
- Transformación de Yeo-Johnson [13]: permite normalizar datos, estabilizar la varianza y reducir el sesgo en una distribución. Es una mejora de la transformación Box-Cox [15], ya que funciona con cualquier número real.

$$outlier(x) = \begin{cases} \text{No es outlier} & Q_1 - 1{,}5 * IQR \leq x \leq Q_3 + 1{,}5 * IQR \\ \text{Si es outlier} & \text{para todo otro caso} \end{cases} \tag{2.2}$$

2.1.2. Escalado de variables

Además de los *outliers*, las magnitudes de las variables de un dataset pueden también afectar el rendimiento de los modelos. Por ejemplo, si una variable tiene un rango de valores entre 0 y 100 y otra entre 0 y 1000, el modelo puede interpretar que la segunda es más importante, lo cual podría ser erróneo.

Para solucionar estos inconvenientes se utilizan técnicas de escalado de variables.

En particular, se utiliza la técnica de *Standard Scaler* [16], cuya fórmula se observa en la ecuación 2.3. Esta transforma los datos de una variable para que tengan una media (μ) de 0 y una desviación estándar (σ) de 1, es decir, asemeja la distribución a la de una normal.

$$z = \frac{x - \mu}{\sigma} \tag{2.3}$$

8 Capítulo 2. Introducción específica

Por este motivo, uno de los procesamientos que se implementan en este trabajo es el de detectar y tratar *outliers*.

Para detectarlos, se utiliza la técnica de IQR o rango intercuartil [16]. Esta mide la dispersión del 50 % central de los datos, es decir, aquellos que se encuentran entre el percentil 25 (cuartil 1 o Q1) y el percentil 75 (cuartil 3 o Q3), y se define mediante la ecuación 2.1.

$$IQR = Q3 - Q1 \tag{2.1}$$

Una vez definido el rango intercuartil, se consideran *outliers* a aquellos valores que se encuentran 1,5 veces el IQR debajo del cuartil 1, o 1,5 veces el IQR sobre el cuartil 3, tal como se define en la ecuación 2.2.

Una vez detectados los *outliers*, se imputan mediante distintas técnicas, entre las que se encuentran:

- Imputación por media: reemplaza los valores atípicos con el valor medio de la distribución.
- Imputación por mediana: reemplaza los valores atípicos con la mediana de la distribución.
- Imputación por moda: reemplaza los valores atípicos con la moda de la distribución.
- Transformación por raíz cuadrada: aplica la función de raíz cuadrada sobre la columna entera.
- Transformación de Yeo-Johnson [15]: permite normalizar datos, estabilizar la varianza y reducir el sesgo en una distribución. Es una mejora de la transformación Box-Cox [17], ya que funciona con cualquier número real.

$$outlier(x) = \begin{cases} \text{No es outlier} & Q_1 - 1{,}5 * IQR \leq x \leq Q_3 + 1{,}5 * IQR \\ \text{Sí es outlier} & \text{para todo otro caso} \end{cases} \tag{2.2}$$

2.1.2. Escalado de variables

Además de los *outliers*, las magnitudes de las variables de un dataset pueden también afectar el rendimiento de los modelos. Por ejemplo, si una variable tiene un rango de valores entre 0 y 100 y otra entre 0 y 1000, el modelo puede interpretar que la segunda es más importante, lo cual podría ser erróneo.

Para solucionar estos inconvenientes se utilizan técnicas de escalado de variables.

En particular, se utiliza la técnica de *Standard Scaler* [18], cuya fórmula se observa en la ecuación 2.3. Esta transforma los datos de una variable para que tengan una media (μ) de 0 y una desviación estándar (σ) de 1, es decir, asemeja la distribución a la de una normal.

$$z = \frac{x - \mu}{\sigma} \tag{2.3}$$

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

2.1.3. Balanceo de dataset

Al trabajar con problemas de clasificación, los datasets implementados tienen una columna *target* u objetivo, la cual describe a una clase a la cual pertenece cada registro. Una de las particularidades que se pueden presentar es la del desbalance: una clase tiene una cantidad mucho mayor de casos que el resto. Estas se conocen como clases dominantes, y su presencia puede afectar el rendimiento de modelos de *machine learning* al hacer que estos ignoren a las clases minoritarias.

Para solucionar este problema, se utilizan técnicas de sobremuestreo u *oversampling*, las cuales agregan registros extras al dataset con el fin de tener un dataset balanceado.

En este trabajo se utilizan las siguientes técnicas:

- SMOTE [17]: permite generar ejemplos sintéticos en función de un caso de la clase minoritaria y sus vecinos más cercanos.
- ADASYN [18]: consiste en una evolución de SMOTE, la cual genera datos sintéticos en áreas donde la densidad de la clase minoritaria es baja y está rodeada de la clase mayoritaria.
- SMOTE-ENN [19]: consiste en una técnicas que comienza aplicando SMOTE, para luego eliminar ejemplos de clases que difieran de sus vecinos mediante ENN (*Edited Nearest Neighbors*.

2.1.4. Selección de features

Otra particularidad de los datasets utilizados en este tipo de trabajo es la cantidad de variables presentes. Estos valores suelen ser muy variados, ya que hay datasets con menos de 10 variables (como el iris dataset [20], que tiene 6 columnas), mientras que otros pueden tener 100 y más columnas.

Para estos últimos casos resulta conveniente aplicar técnicas de selección de características o *features*. Estas nos permiten filtrar algunas columnas con el fin de quedarse con las más importantes. De esta manera, se elimina cierto ruido de datos pocos relevantes, como así también se reduce el sobreajuste (u *overfitting*) al entrenar modelos.

En particular, en este trabajo se hace uso de la técnica de información mutua o *mutual information* [21]. Esta permite permite cuantificar cuánta información se obtiene sobre la variable objetivo al utilizar una característica en particular, es decir, que tan dependientes son entre sí. A mayor valor, mayor es la información que una *feature* nos proporciona sobre la variable objetivo. Por eso, se busca obtener la información mutua de todas las columnas frente al *target*, luego se hace un ranking ordenando de mayor a menor, y finalmente se selecciona un subconjunto de las mejores variables de dicho ranking.

2.2. Modelos de inteligencia artificial

En los trabajos de clasificación mediante datasets, los primeros pasos y tareas suelen dedicarse a la preparación del dataset, utilizando técnicas como las mencionadas anteriormente. Una vez preparado, los datasets son utilizados como entrada

2.2. Modelos de inteligencia artificial 9

2.1.3. Balanceo de dataset

Al trabajar con problemas de clasificación, los datasets implementados tienen una columna *target* u objetivo, la cual describe a una clase a la cual pertenece cada registro. Una de las particularidades que se pueden presentar es la del desbalance: una clase tiene una cantidad mucho mayor de casos que el resto. Estas se conocen como clases dominantes, y su presencia puede afectar el rendimiento de modelos de *machine learning* al hacer que estos ignoren a las clases minoritarias.

Para solucionar este problema, se utilizan técnicas de sobremuestreo u *oversampling*, las cuales agregan registros extras al dataset con el fin de tener un dataset balanceado.

En este trabajo se utilizan las siguientes técnicas:

- SMOTE [19]: permite generar ejemplos sintéticos en función de un caso de la clase minoritaria y sus vecinos más cercanos.
- ADASYN [20]: consiste en una evolución de SMOTE, la cual genera datos sintéticos en áreas donde la densidad de la clase minoritaria es baja y está rodeada de la clase mayoritaria.
- SMOTE-ENN [21]: consiste en una técnicas que comienza aplicando SMOTE, para luego eliminar ejemplos de clases que difieran de sus vecinos mediante ENN (*Edited Nearest Neighbors*).

2.1.4. Selección de features

Otra particularidad de los datasets utilizados en este tipo de trabajo es la cantidad de variables presentes. Estos valores suelen ser muy variados, ya que hay datasets con menos de 10 variables (como el iris dataset [22], que tiene 6 columnas), mientras que otros pueden tener 100 y más columnas.

Para estos últimos casos resulta conveniente aplicar técnicas de selección de características o *features*. Estas nos permiten filtrar algunas columnas con el fin de quedarse con las más importantes. De esta manera, se elimina cierto ruido de datos pocos relevantes, como así también se reduce el sobreajuste (u *overfitting*) al entrenar modelos.

En particular, en este trabajo se hace uso de la técnica de información mutua o *mutual information* [23]. Esta permite permite cuantificar cuánta información se obtiene sobre la variable objetivo al utilizar una característica en particular, es decir, que tan dependientes son entre sí. A mayor valor, mayor es la información que una *feature* nos proporciona sobre la variable objetivo. Por eso, se busca obtener la información mutua de todas las columnas frente al *target*, luego se hace un ranking ordenando de mayor a menor, y finalmente se selecciona un subconjunto de las mejores variables de dicho ranking.

2.2. Modelos de inteligencia artificial

En los trabajos de clasificación mediante datasets, los primeros pasos y tareas suelen dedicarse a la preparación del dataset, utilizando técnicas como las mencionadas anteriormente. Una vez preparado, los datasets son utilizados como entrada

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

de distintos algoritmos de *machine learning* con el fin de obtener un modelo. Estos modelos posteriormente nos permiten realizar predicciones o clasificaciones a partir de datos particulares de entrada.

En este trabajo, se implementan los siguientes modelos de *machine learning*:

- Regresión logística o *logistic regression* [7]: permite obtener la probabilidad de pertenencia a una de las clases de estudio. A partir de dicha probabilidad se determina a qué clase pertenece una muestra en particular.
- SVM o *support vector machine* [6]: se caracteriza por buscar un hiperplano que separe las clases de estudio con el margen más amplio posible.
- XGBoost [8]: consiste en un modelo de ensambles basado en árboles de decisión. Estos árboles son entrenados de manera secuencial, especializándose en corregir los errores del árbol anterior. Esto permite que sea un modelo robusto al evitar el sobreajuste.

2.3. Optimización de hiperparámetros

Los modelos de *machine learning* se caracterizan por tener hiperparámetros: variables de configuración que se definen antes de comenzar el entrenamiento. Estas se diferencian de un parámetro ya que controlan como se comporta el algoritmo.

A partir de esto, surge la necesidad de optimizar estos hiperparámetros con el fin de encontrar un equilibrio óptimo entre sesgo y varianza, lo cual a su vez permite evitar el sobreajuste (*overfitting*) o subajuste (*underfitting* del modelo.

En este trabajo se utiliza el *framework* optuna [22]: una biblioteca de código abierto basada en python. Esta utiliza una optimización bayesiana para, en lugar de probar valores al azar, aprender de intentos anteriores y enfocarse en las combinaciones de hiperparámetros más prometedoras, encontrando así las más óptimas.

2.4. Herramientas - Tracking server

Al trabajar con distintos modelos de *machine learning* resulta crucial hacer un seguimiento de los mismos, enfocándose en los datos o datasets de entrada utilizados, los hiperparámetros configurados, las métricas y gráficos obtenidas luego del entrenamiento, entre otras cosas.

En este trabajo se utiliza MLFlow [23], una herramienta de código abierto que nos permite hacer este seguimiento y registro de configuraciones, a fin de utilizar tanto estos valores como los modelos en otras etapas del trabajo.

2.5. Herramientas para automatización

Este trabajo consta de distintas etapas, tales como el preprocesamiento de dataset, entrenamiento de modelos, optimización de hiperparámetros, obtención de métricas, entre otras.

El trabajo realizada en ellas se automatiza mediante la herramienta de código abierto Airflow [24], la cual nos permite programar, automatizar y monitorear los flujos de trabajo o *workflows* mencionados.

10 Capítulo 2. Introducción específica

de distintos algoritmos de *machine learning* con el fin de obtener un modelo. Estos modelos posteriormente nos permiten realizar predicciones o clasificaciones a partir de datos particulares de entrada.

En este trabajo, se implementan los siguientes modelos de *machine learning*:

- Regresión logística o *logistic regression* [9]: permite obtener la probabilidad de pertenencia a una de las clases de estudio. A partir de dicha probabilidad se determina a qué clase pertenece una muestra en particular.
- SVM o *support vector machine* [8]: se caracteriza por buscar un hiperplano que separe las clases de estudio con el margen más amplio posible.
- XGBoost [10]: consiste en un modelo de ensambles basado en árboles de decisión. Estos árboles son entrenados de manera secuencial, especializándose en corregir los errores del árbol anterior. Esto permite que sea un modelo robusto al evitar el sobreajuste.

2.3. Optimización de hiperparámetros

Los modelos de *machine learning* se caracterizan por tener hiperparámetros: variables de configuración que se definen antes de comenzar el entrenamiento. Estas se diferencian de un parámetro ya que controlan como se comporta el algoritmo.

A partir de esto, surge la necesidad de optimizar estos hiperparámetros con el fin de encontrar un equilibrio óptimo entre sesgo y varianza, lo cual a su vez permite evitar el sobreajuste (*overfitting*) o subajuste (*underfitting* del modelo.

En este trabajo se utiliza el *framework* optuna [24]: una biblioteca de código abierto basada en python. Esta utiliza una optimización bayesiana para, en lugar de probar valores al azar, aprender de intentos anteriores y enfocarse en las combinaciones de hiperparámetros más prometedoras, encontrando así las más óptimas.

2.4. Herramientas - Tracking server

Al trabajar con distintos modelos de *machine learning* resulta crucial hacer un seguimiento de los mismos, enfocándose en los datos o datasets de entrada utilizados, los hiperparámetros configurados, las métricas y gráficos obtenidas luego del entrenamiento, entre otras cosas.

En este trabajo se utiliza MLFlow [25], una herramienta de código abierto que nos permite hacer este seguimiento y registro de configuraciones, a fin de utilizar tanto estos valores como los modelos en otras etapas del trabajo.

2.5. Herramientas para automatización

Este trabajo consta de distintas etapas, tales como el preprocesamiento de dataset, entrenamiento de modelos, optimización de hiperparámetros, obtención de métricas, entre otras.

El trabajo realizada en ellas se automatiza mediante la herramienta de código abierto Airflow [26], la cual nos permite programar, automatizar y monitorear los flujos de trabajo o *workflows* mencionados.

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

Estos *workflows* se organizan mediante DAGs[1], o grafo acíclico dirigido por sus siglas en inglés, los cuales siguen un orden lógico (secuencial o en paralelo) sin bucles para lograr un objetivo. De esta forma, se pueden orquestar tareas y escalar procesos de manera automática.

2.6. Desarrollo de API y herramientas para su consumo

En este trabajo se presenta una herramienta web que permite realizar predicciones sobre la quiebra de una empresa. Esta consiste en un API Rest, una interfaz de programación de aplicaciones cuya arquitectura, basada en REST [25], permite la comunicación e intercambio de datos mediante el protocolo HTTP. Esto permite que el servicio web sea escalable, ya que las distintas peticiones de clientes se pueden distribuir entre distintas instancias de un mismo servicio; y simple, ya que tiene métodos HTTP o *endpoints* para operaciones como predicción, recuperación de datos, etc.;

Para desarrollarlo, se usa el framework FastAPI [26], el cual permite programar las API utilizando código python, brindando simplicidades tales como la documentación automática, el soporte de métodos asincrónicos, validación automática de datos de entrada, entre otras.

Para validar el correcto funcionamiento de esta API se utilizan herramientas como curl [12] y postman [27], las cuales permiten realizar peticiones web a los distintos *endpoints* disponibles, tanto de manera manual como automática.

2.7. Uso de contenedores

Las distintas herramientas mencionadas se pueden utilizar de diferentes maneras. Una de estas es mediante el uso de contenedores.

Un contenedor o *container* es una unidad de software que empaqueta a una aplicación y a todas sus dependencias a fin de que se ejecute de forma rápida en cualquier entorno. Es decir, permite que un software sea portable, ya que se puede utiliar tanto de manera local como en la nube; aislado, ya que varias aplicaciones se pueden ejecutar en el mismo entorno sin afectarse entre sí; y fácil de desplegar.

El uso de contenedores en este trabajo se usan contenedores mediante la herramienta Docker [28].

[1] https://www.astronomer.io/docs/learn/dags

2.6. Desarrollo de API y herramientas para su consumo 11

Estos *workflows* se organizan mediante DAGs[1], o grafo acíclico dirigido por sus siglas en inglés, los cuales siguen un orden lógico (secuencial o en paralelo) sin bucles para lograr un objetivo. De esta forma, se pueden orquestar tareas y escalar procesos de manera automática.

2.6. Desarrollo de API y herramientas para su consumo

En este trabajo se presenta una herramienta web que permite realizar predicciones sobre la quiebra de una empresa. Esta consiste en un API Rest, una interfaz de programación de aplicaciones cuya arquitectura, basada en REST [27], permite la comunicación e intercambio de datos mediante el protocolo HTTP. Esto permite que el servicio web sea escalable, ya que las distintas peticiones de clientes se pueden distribuir entre distintas instancias de un mismo servicio; y simple, ya que tiene métodos HTTP o *endpoints* para operaciones como predicción, recuperación de datos, etc.;

Para desarrollarlo, se usa el framework FastAPI [28], el cual permite programar las API utilizando código python, brindando simplicidades tales como la documentación automática, el soporte de métodos asincrónicos, validación automática de datos de entrada, entre otras.

Para validar el correcto funcionamiento de esta API se utilizan herramientas como curl [14] y postman [29], las cuales permiten realizar peticiones web a los distintos *endpoints* disponibles, tanto de manera manual como automática.

2.7. Uso de contenedores

Las distintas herramientas mencionadas se pueden utilizar de diferentes maneras. Una de estas es mediante el uso de contenedores.

Un contenedor o *container* es una unidad de software que empaqueta a una aplicación y a todas sus dependencias a fin de que se ejecute de forma rápida en cualquier entorno. Es decir, permite que un software sea portable, ya que se puede utiliar tanto de manera local como en la nube; aislado, ya que varias aplicaciones se pueden ejecutar en el mismo entorno sin afectarse entre sí; y fácil de desplegar.

El uso de contenedores en este trabajo se usan contenedores mediante la herramienta Docker [30].

[1] https://www.astronomer.io/docs/learn/dags

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

Bibliografía

[1] Konstantin Danilov. «Corporate Bankruptcy: Assessment, Analysis and Prediction of Financial Distress, Insolvency, and Failure». En: *SSRN Electronic Journal* (sep. de 2014). DOI: 10.2139/ssrn.2467580.

[2] Edward I. Altman. «Financial Ratios, Discriminant Analysis and the Prediction of Corporate Bankruptcy». En: *The Journal of Finance* 23.4 (1968), págs. 589-609. ISSN: 00221082, 15406261. URL: http://www.jstor.org/stable/2978933 (visitado 28-03-2026).

[3] Kamil Samara y Apurva Shinde. «Bankruptcy Prediction Using Machine Learning and Data Preprocessing Techniques». En: *Analytics* 4 (sep. de 2025), pág. 22. DOI: 10.3390/analytics4030022.

[4] Taiwan Economic Journal. *Taiwanese Bankruptcy Prediction*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5004D. 2020.

[5] Haoming Wang y Xiangdong Liu. «Undersampling bankruptcy prediction: Taiwan bankruptcy data». En: *PLOS ONE* 16.7 (jul. de 2021), págs. 1-17. DOI: 10.1371/journal.pone.0254030. URL: https://doi.org/10.1371/journal.pone.0254030.

[6] Theodoros Evgeniou y Massimiliano Pontil. «Support Vector Machines: Theory and Applications». En: *Support Vector Machines: Theory and Applications*. Vol. 2049. Sep. de 2001, págs. 249-257. ISBN: 978-3-540-42490-1. DOI: 10.1007/3-540-44673-7_12.

[7] Moo K. Chung. *Introduction to logistic regression*. 2020. arXiv: 2008.13567 [stat.ME]. URL: https://arxiv.org/abs/2008.13567.

[8] Tianqi Chen y Carlos Guestrin. «XGBoost: A Scalable Tree Boosting System». En: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. ACM, ago. de 2016, 785–794. DOI: 10.1145/2939672.2939785. URL: http://dx.doi.org/10.1145/2939672.2939785.

[9] *Azure*. https://azure.microsoft.com/en-us.

[10] *AWS*. https://aws.amazon.com/.

[11] *Google Cloud*. https://cloud.google.com/.

[12] *cURL*. https://curl.se/.

[13] In-Kwon Yeo y Richard A. Johnson. «A new family of power transformations to improve normality or symmetry». En: *Biometrika* 87.4 (dic. de 2000), págs. 954-959. ISSN: 0006-3444. DOI: 10.1093/biomet/87.4.954. eprint: https://academic.oup.com/biomet/article-pdf/87/4/954/633221/870954.pdf. URL: https://doi.org/10.1093/biomet/87.4.954.

[14] Vaibhav Tummalapalli. «Outlier Detection and Treatment for Machine Learning Models». En: 2 (nov. de 2025). DOI: 10.5281/zenodo.16500050.

[15] G. E. P. Box y D. R. Cox. «An Analysis of Transformations». En: *Journal of the Royal Statistical Society: Series B (Methodological)* 26.2 (1964), págs. 211-243. DOI: https://doi.org/10.1111/j.2517-6161.1964.tb00553.x. eprint: https://rss.onlinelibrary.wiley.com/doi/pdf/10.1111/j.2517-6161.1964.tb00553.x.

Capítulo 3

Diseño e implementación

En este capítulo se comienza describiendo el flujo de trabajo utilizado, explicando la diferencia entre los ambientes locales y dockerizados. Luego, se describe la arquitectura de cada uno de estos ambientes. Después, se detallan los pasos realizados para preprocesar datos, como así también para entrenar y optimizar los hiperparámetros de los modelos. Posteriormente, se explica como se configuran el *tracking server* y Airflow. Finalmente, se describe el diseño del servicio web que permite realizar predicciones.

3.1. Flujo de trabajo

En este trabajo se utilizó un flujo de trabajo que abarca desde la obtención de los datos base hasta la publicación de un servicio web, y puede ser resumido en gran medida por la figura 3.1.

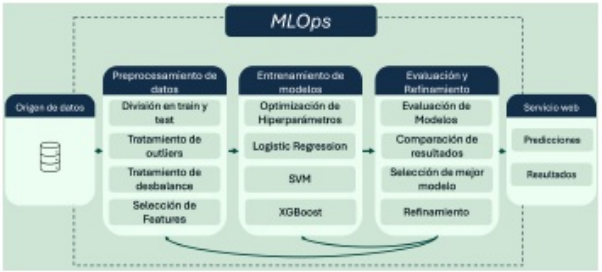

FIGURA 3.1. Flujo de trabajo utilizado.

3.1.1. Etapas del flujo de trabajo

La primer etapa del flujo de trabajo implementado consta de la obtención del origen de datos del sistema, siendo en este caso el dataset de *Taiwanese Bankruptcy Prediction* [4], el cual es de origen público y cuenta con más de 6800 registros, los cuales tienen información financiera de empresas del mercado de Taiwán entre 1999 y 2009. La última etapa consiste en la publicación de un servicio web, el cual permitirá a los distintos usuarios realizar predicciones utilizando un modelo entrenado en este trabajo.

DiffPDF • C:/fiuba/Plantilla-para-memoria/Otros docs/pdfs/TI_Acevedo_Zain_Gaspar_V2.pdf vs. C:/fiuba/Plantilla-para-memoria/Otros

URL: https://rss.onlinelibrary.wiley.com/doi/abs/10.1111/j.2517-6161.1964.tb00553.x.

[16] *Standard Scaler*. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html.

[17] Nitesh Chawla et al. «SMOTE: Synthetic Minority Over-sampling Technique». En: *Journal of Artificial Intelligence Research* 16 (jun. de 2002), págs. 321-357. DOI: 10.1613/jair.953.

[18] Haibo He et al. «ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning». En: *Proceedings of the International Joint Conference on Neural Networks* (jul. de 2008), págs. 1322 -1328. DOI: 10.1109/IJCNN.2008.4633969.

[19] Arnab Mukherjee et al. «SMOTE-ENN resampling technique with Bayesian optimization for multi-class classification of dry bean varieties». En: *Applied Soft Computing* 181 (2025), pág. 113467. ISSN: 1568-4946. DOI: https://doi.org/10.1016/j.asoc.2025.113467. URL: https://www.sciencedirect.com/science/article/pii/S1568494625007781.

[20] *Iris dataset*. https://www.kaggle.com/datasets/uciml/iris.

[21] Hanchuan Peng, Fuhui Long y C. Ding. «Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy». En: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 27.8 (2005), págs. 1226-1238. DOI: 10.1109/TPAMI.2005.159.

[22] *Optuna*. https://optuna.org/.

[23] *MLFlow*. https://mlflow.org/.

[24] *Apache Airflow*. https://airflow.apache.org/.

[25] Carlos Rodriguez et al. «REST APIs: A Large-Scale Analysis of Compliance with Principles and Best Practices». En: jun. de 2016, págs. 21-39. ISBN: 978-3-319-38790-1. DOI: 10.1007/978-3-319-38791-8_2.

[26] *FastAPI*. https://fastapi.tiangolo.com/.

[27] *postman*. https://www.postman.com/.

[28] *Docker*. https://www.docker.com/.

Entre estas dos etapas se encuentran otras que se caracterizan por ejecutarse de manera iterativa. Esto se debe a que se busca mejorar los resultados de las iteraciones anteriores, ya sea modificando algunos de sus parámetros o alterando la cantidad y/o orden de sub-tareas dentro de cada etapa. Estas son:

- Preprocesamiento de datos: en donde, a grandes razgos, se mejora la calidad de dataset. Entre sus sub-tareas se encuentran la división del dataset en train y test, tratamiento de *outliers*, tratamiento de desbalance y selección de *features*.
- Entrenamiento de modelos: en donde se crean, entrenan y optimizan modelos de *machine learning*, utilizados para realizar predicciones de quiebra.
- Evaluación y refinamiento de modelos: en donde se evalúan los modelos de la etapa anterior y se selecciona al mejor de ellos.

3.1.2. Implementación del flujo de trabajo

Para implementar el flujo de trabajo se optó por definir dos ambientes: un ambiente local y otro dockerizado.

El ambiente local se utiliza principalmente para experimentar distintas técnicas, herramientas e incluso sus posibles combinaciones. Por este motivo, este ambiente está constituído por *notebooks* de python. Una vez determinada cuáles son las mejores técnicas y en que orden deben ejecutarse, se procede a replicarlas en el ambiente dockerizado.

Por su parte, el ambiente dockerizado está orientado a la automatización del flujo de trabajo. De ahí a que se utilicen distintos contenedores de docker. En este caso, se crean contenedores para las etapas desde la descarga del dataset hasta la exposición de los modelos mediante un servicio web.

3.2. Arquitectura del sistema

En la sección anterior se describe el por qué del uso de dos ambientes al implementar el flujo de trabajo. En esta sección se describe la arquitectura de cada uno de estos ambientes. Igualmente, se remarca que en las futuras secciones y capítulos, al hablar de "sistema"se hará referencia al sistema desplegado mediante el ambiente dockerizado.

3.2.1. Arquitectura del ambiente local

El ambiente local se caracteriza por estar compuesto por un total de cinco directorios, cada uno dedicado a explorar distintas técnicas y/o herramientas. Cada directorio tiene una cantidad distintas de notebooks de python, las cuales se ejecutan en el orden indicado en la figura 3.2. Por su parte, las notebooks generan dos datasets, uno para los datos de entrenamiento o *train*, y otro para los datos de pruebas o *test*. Estos datasets son a su vez utilizados como inputs por las notebooks del directorio siguiente en la secuencia de ejecución.

El primer directorio que se tiene en el ambiente local es el de análisis exploratorio. Éste, mediante la única notebook presente, se encarga de descargar el dataset original con el fin de analizarlo. En esta notebook se exploran las *features* del dataset y su columna *target* del dataset, haciendo foco en sus distribuciones, búsqueda de